

IAplicada

Teste Prático — Dev No-Code Junior

Processo Seletivo 2026

Prazo de entrega:	48 horas a partir do recebimento deste documento
Formato de entrega:	Links funcionando + documento com prints e explicações (detalhado na seção 5)
Ferramentas:	Frontend livre (Lovable, Bolt, v0, ou outra) + Supabase (obrigatório) + n8n (automação) + GitHub (obrigatório)
Nível esperado:	Junior — não esperamos perfeição, esperamos raciocínio, estrutura e entrega funcional

1. Contexto

A IAplicada constrói sistemas de gestão sob medida para empresas. Nosso stack principal combina Supabase (backend e banco de dados), ferramentas de frontend no-code/low-code (Lovable, Bolt, v0, entre outras), n8n (automações e integrações), e GitHub pra versionamento.

Neste teste, você vai construir um sistema completo para um cliente fictício. Não é exercício de tutorial. É um cenário que simula o tipo de entrega que fazemos de verdade. Queremos ver como você modela dados, como conecta as peças, como pensa em fluxos, e se consegue entregar algo funcional de ponta a ponta.

Pode usar IA (Claude, Cursor, ChatGPT) como ferramenta de apoio. **Mas o sistema precisa funcionar de verdade e você precisa entender o que construiu.** Na entrevista, vamos abrir o sistema juntos, navegar pelas telas, olhar o banco, e pedir pra você explicar cada decisão.

2. O Cenário

A "SmartLar" é uma empresa de automação residencial. Ela vende e instala equipamentos inteligentes em casas e apartamentos (fechaduras digitais, câmeras, sensores, assistentes de voz, iluminação inteligente). Hoje o dono, Rafael, gerencia tudo no WhatsApp e num caderninho. O resultado:

- Ele esquece orçamentos que mandou e perde vendas
- Não sabe quais instalações estão pendentes na semana
- Não tem visão de quanto já faturou no mês e quanto ainda tem a receber
- Os dois técnicos da equipe (Lucas e Pedro) não sabem a agenda deles sem ligar pro Rafael
- Clientes ligam perguntando status e ele tem que procurar no WhatsApp

O Rafael contratou a IAplicada pra resolver isso. Seu trabalho é entregar o sistema.

3. O Que Você Vai Construir

3.1 Banco de Dados (Supabase)

Crie um projeto gratuito no Supabase. Modele o banco pensando em como o sistema vai funcionar na prática. As tabelas abaixo são o mínimo esperado, mas você tem liberdade pra adicionar campos que julgar necessários.

Tabela	Campos mínimos	Regras de negócio
clientes	id, nome, telefone, email, endereco, created_at	Telefone obrigatório (WhatsApp). Endereço é onde vai ser a instalação.
tecnicos	id, nome, telefone, especialidade	Já cadastre 2 técnicos: Lucas (câmeras e sensores) e Pedro (fechaduras e iluminação).
produtos	id, nome, categoria, preco_unitario, descricao	Cadastre pelo menos 6 produtos em 3 categorias diferentes (ex: segurança, iluminação, automação).
pedidos	id, cliente_id, tecnico_id, status, data_instalacao, valor_total, forma_pagamento, observacoes, created_at	Status segue o fluxo: orcamento → aprovado → agendado → em_andamento → concluido → cancelado. Nem todo status pode ir pra qualquer outro (veja regras abaixo).
itens_pedido	id, pedido_id, produto_id, quantidade, preco_unitario, subtotal	Um pedido pode ter vários produtos. O valor_total do pedido é a soma dos subtotais.

Fluxo de status do pedido (importante):

orcamento → aprovado → agendado → em_andamento → concluido

orcamento → cancelado (pode cancelar direto do orçamento)

aprovado → cancelado (pode cancelar depois de aprovar)

Não pode voltar de status. Não pode ir de "orcamento" direto pra "em_andamento", por exemplo.

O que avaliamos aqui: se a modelagem faz sentido e sustenta o sistema. Se as relações (foreign keys) estão corretas. Se os tipos de dados estão adequados. Se você pensou em campos que o sistema precisa pra funcionar (não só os que listamos). Se tem dados de exemplo suficientes pra testar.

Bônus (não obrigatório): RLS (Row Level Security) configurado. Tabela de historico_status pra registrar quando cada mudança de status aconteceu.

3.2 Frontend (ferramenta livre)

Use a ferramenta de frontend que preferir: Lovable, Bolt, v0, ou qualquer outra. O importante é que o sistema esteja conectado ao Supabase e funcionando. O código precisa estar num repositório GitHub (obrigatório). O sistema precisa ter as telas abaixo, todas funcionais e conectadas ao banco.

Tela 1 — Dashboard do Rafael

É a primeira coisa que o Rafael vê ao abrir o sistema. Precisa mostrar:

— Indicadores no topo: total de pedidos do mês, valor total faturado (pedidos concluídos), valor a receber (pedidos aprovados + agendados + em andamento), pedidos pendentes de agendamento

— Lista das próximas instalações agendadas (próximos 7 dias) com: cliente, endereço, técnico responsável, data

— Lista de orçamentos aguardando aprovação

Tela 2 — Cadastro e lista de clientes

Formulário pra cadastrar novo cliente (nome, telefone, email, endereço). Lista de todos os clientes cadastrados com busca por nome ou telefone. Ao clicar num cliente, ver os pedidos dele.

Tela 3 — Catálogo de produtos

Lista dos produtos disponíveis, organizados por categoria. Possibilidade de cadastrar novo produto e editar preço de um existente.

Tela 4 — Criar novo pedido (fluxo completo)

Essa é a tela mais importante. O fluxo de criação do pedido:

- Selecionar cliente (dos já cadastrados) ou cadastrar um novo na hora
- Adicionar produtos ao pedido: selecionar produto, definir quantidade, ver subtotal calculado automaticamente
- Poder adicionar múltiplos produtos ao mesmo pedido
- Ver valor total do pedido atualizado conforme adiciona itens
- Adicionar observações (ex: "portão eletrônico antigo, verificar compatibilidade")
- Salvar como "orçamento" (status inicial)

Tela 5 — Gestão de pedidos

Lista de todos os pedidos com filtros por status. Ao abrir um pedido, ver todos os detalhes (cliente, produtos, valores, técnico, datas). Possibilidade de avançar o status do pedido (seguindo o fluxo definido). Ao mudar pra "agendado", obrigar a selecionar técnico e data de instalação.

Tela 6 — Agenda dos técnicos

Visualização por técnico: seleciona o técnico e vê as instalações agendadas dele, com data, cliente e endereço. O técnico precisa conseguir marcar uma instalação como "em andamento" e depois "concluída" direto dessa tela.

O que avaliamos aqui: se o sistema funciona de ponta a ponta (cadastra cliente, cria pedido com itens, gerencia status, mostra no dashboard). Se os cálculos estão corretos (subtotais, valor total). Se o fluxo de status é respeitado. Se os dados vêm e vão pro Supabase de verdade. Se a interface é usável (não precisa ser bonita, precisa funcionar e fazer sentido pro Rafael).

Bônus: tela de login com autenticação Supabase (não obrigatório, mas mostra maturidade). Deploy funcional (Vercel, Netlify, ou link público da ferramenta).

3.3 Automações (n8n)

Crie uma conta gratuita no n8n (n8n.cloud) e construa as automações abaixo. Todas precisam funcionar de verdade, não só em teste manual.

Automação 1 — Notificação de novo pedido (obrigatória)

Quando um novo pedido é criado no Supabase (status = orçamento), o n8n deve capturar esse evento e registrar a informação em algum destino externo. Pode ser: enviar um email, salvar numa linha do Google Sheets, postar num webhook de teste (webhook.site), ou qualquer destino que prove que a automação disparou com os dados corretos (nome do cliente, valor total, data).

Automação 2 — Alerta de instalação agendada (obrigatória)

Todo dia (pode ser via cron no n8n), verificar no Supabase se existem instalações agendadas para o dia seguinte. Se existirem, enviar um resumo (pode ser email, Google Sheets, ou webhook) com: nome do cliente, endereço, técnico responsável, horário.

Automação 3 — Bônus (não obrigatória)

Quando um pedido muda de status pra "concluído", registrar o faturamento em algum lugar (Google Sheets com data, cliente, valor, forma de pagamento). Isso simularia o início de um controle financeiro.

O que avaliamos aqui: se você sabe configurar triggers no n8n que escutam eventos do Supabase. Se os workflows funcionam em produção (não só clicando "test" manualmente). Se os dados chegam corretos no destino. Se pensou em tratamento de erro (o que acontece se o trigger falhar? se não tiver agendamento pro dia seguinte?). Se a lógica da Automação 2 realmente consulta o banco e filtra corretamente.

4. Regras Importantes

Dados de exemplo:

Popule o sistema com dados realistas pra gente conseguir testar. No mínimo: 5 clientes cadastrados, 6 produtos em 3 categorias, 8 pedidos em diferentes status (alguns em orçamento, alguns agendados, alguns concluídos, um cancelado). Sem dados de exemplo, não conseguimos avaliar se o sistema funciona.

Cálculos precisam funcionar:

Se o pedido tem 2x Câmera IP (R\$ 450 cada) + 1x Sensor de presença (R\$ 180), o valor total precisa ser R\$ 1.080. Se esse cálculo não bate, é problema sério.

Fluxo de status precisa ser respeitado:

O sistema não pode permitir que um pedido em "orçamento" pule direto pra "concluído". Se o frontend não trava isso, o candidato precisa pelo menos documentar que sabe que essa regra existe.

Tudo conectado:

O frontend precisa estar conectado ao Supabase. Dados cadastrados no frontend precisam aparecer no banco, e vice-versa. Automações precisam disparar a partir de dados reais do banco, não de dados hardcoded no n8n.

5. O Que Entregar

Envie um documento (Google Docs, Notion, ou PDF) contendo:

#	Entregável	Detalhe
---	------------	---------

1	Link do projeto funcionando	Precisa abrir e funcionar. Vamos navegar por todas as telas e testar os fluxos.
2	Link ou prints do Supabase	Queremos ver: tabelas criadas, relações entre elas, dados de exemplo populados.
3	Prints dos workflows n8n + evidências	Screenshot de cada workflow + print do log de execução mostrando que rodou com sucesso.
4	Explicação das suas decisões técnicas	Por que modelou o banco assim? Como resolveu os cálculos? Como conectou n8n com Supabase? O que faria diferente com mais tempo?
5	Onde usou IA e pra quê	Seja honesto. Queremos saber se você usa IA como ferramenta (bom) ou se só copiou e colou sem entender (ruim).
6	Link do repositório GitHub (obrigatório)	Repositório público com o código do projeto. Queremos ver commits, organização e histórico.

6. Como Vamos Avaliar

Não esperamos perfeição de um junior. Esperamos raciocínio claro, entrega funcional e honestidade sobre o que sabe e o que não sabe.

Critério	Peso	O que olhamos
Modelagem de dados	20%	Tabelas fazem sentido? Relações corretas? Tipos de dados adequados? Campos suficientes? Dados de exemplo?
Frontend funcional	30%	6 telas funcionam? Fluxo de pedido completo funciona (criar, adicionar itens, calcular total)? Filtros e buscas? Interface usável?
Lógica de negócio	15%	Cálculos de valor corretos? Fluxo de status respeitado? Validações básicas (campos obrigatórios)?
Automações	15%	As 2 automações obrigatórias funcionam de verdade? Triggers corretos? Dados chegam completos? Tratamento de erro?
Integração ponta a ponta	10%	Front conecta com back? Automação conecta com banco? Tudo funciona junto como um sistema?
Documentação e raciocínio	10%	Explicou decisões? Sabe o que fez? Identificou limitações? Foi honesto sobre uso de IA?

7. O Que Elimina

Estes pontos resultam em reprovação automática:

- Sistema não funciona (link quebrado, erro ao abrir, dados não carregam)
- Não tem banco de dados real (tudo hardcoded no front, sem Supabase)
- Nenhuma automação funcional no n8n
- Pedido não calcula valores corretamente
- Fluxo de criação de pedido não funciona (não consegue adicionar itens)
- Não tem repositório no GitHub ou repositório vazio
- Cópia de tutorial da internet sem adaptação ao cenário do teste
- Na entrevista, não sabe explicar o que construiu

— Não entregou dentro de 48 horas sem avisar

8. Dicas

Comece pelo banco. Se a modelagem estiver errada, todo o resto vai sofrer. Pense nas relações antes de criar as tabelas.

Depois do banco, vá pro frontend. Comece pelas telas mais simples (cadastro de cliente, lista de produtos) antes de atacar o fluxo de pedido.

Deixe as automações pro final. Elas dependem de ter dados reais no banco e o frontend funcionando pra testar.

Teste o fluxo completo antes de entregar: cadastre um cliente novo, crie um pedido com 3 produtos, avance o status até concluído, e verifique se tudo refletiu no dashboard e se as automações dispararam.

Se algo não funcionou, explique o que tentou. Preferimos honestidade a entrega maquiada.

Se travou em algum ponto, documente onde travou e o que faria se tivesse mais tempo.

Não gaste tempo em design. Interface limpa e funcional vale mais que bonita e quebrada.

Boa sorte! Estamos ansiosos pra ver o que você constrói.

Qualquer dúvida sobre o enunciado, pode perguntar antes de começar.

IAplicada — Processo Seletivo Dev No-Code Junior